**The Australian National University**
2600 ACT | Canberra | Australia

Australian National University

**School of Computing**

College of Engineering, Computing and Cybernetics (CECC)

# Event Model Learning from Story-Based Traces

— 12 pt research project (S1/S2 2024)

A report submitted for the course
*COMP4560, Advanced Computing Project*

**By:**
Ryan Budianto

**Supervisor:**
Dr. Patrik Haslum

October 2024

## Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the University Academic Misconduct Rules;
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

October, Ryan Budianto

# Acknowledgements

I would like to express my gratitude to my supervisor, Dr. Patrik Haslum, for giving me the opportunity to undertake this project. After struggling for months to find a supervisor, his willingness to support my research is sincerely appreciated.

I would also like to thank the faculty and staff at the ANU College of Engineering and Computer Science for providing a supportive academic environment and the resources necessary to complete this work.

Finally, I am grateful to my family and friends for their continuous encouragement and support throughout this journey.

# Abstract

This project successfully adapts the LOCM2 algorithm to learn event models from story-based traces, overcoming challenges such as nested events, uncertain object types, and partial event ordering. By developing a robust data processing pipeline and implementing significant algorithmic modifications, the enhanced LOCM2 effectively handles intricate narrative structures. The algorithm processed a dataset of 3,487 plans, generating 165,364 Finite State Machines (FSMs) with a total of 425,022 states and 327,168 transitions in under six seconds. The average FSM complexity was approximately 2.57 states and 1.98 transitions per FSM. Despite effectively managing nested events and probabilistic object type distributions, the project identified ongoing concerns regarding scalability for larger and more diverse datasets. Future work will focus on integrating the learned FSMs into story generation systems, conducting comprehensive evaluations against state-of-the-art methods, and optimizing the algorithm for enhanced scalability. Overall, this project lays a foundation for advancing automated story generation through effective event model learning.

# Table of Contents


**1 Introduction** **1**
1.1 Background and Motivation . . . 2
1.2 Objectives and Scope . . . 3
1.3 Project Overview . . . 3
1.4 Contributions . . . 4
1.5 Report Structure . . . 4

**2 Literature Review** **7**
2.1 Overview of Event Model Learning . . . 7
2.2 LOCM and LOCM2 Algorithms . . . 7
2.2.1 Learning Object-Centric Models (LOCM) . . . 7
2.2.2 Limitations of LOCM . . . 8
2.2.3 LOCM2 Enhancements . . . 8
2.2.4 Applications and Limitations . . . 9
2.3 Challenges with Story-Based Traces . . . 9
2.3.1 Complex Arguments and Nested Events . . . 9
2.3.2 Partial and Uncertain Traces . . . 10
2.3.3 Uncertain Object Types and Dynamic Typing . . . 10
2.3.4 Comparison to Classical Planning Traces . . . 10
2.4 Summary . . . 11

**3 Methodology** **13**
3.1 Dataset Preparation . . . 13
3.1.1 Source of Data . . . 13
3.1.2 Data Structure . . . 13
3.1.3 Preprocessing Steps . . . 14
3.2 Adaptation of the LOCM2 Algorithm . . . 14
3.2.1 Understanding LOCM2 . . . 14
3.2.2 Identified Limitations . . . 15
3.2.3 Modifications Made . . . 15
3.3 Handling Complexities in Story-Based Data . . . 16
3.3.1 Partial Orders in Events . . . 16

3.3.2 Uncertain Object Types . . . 16
3.3.3 Nested Events . . . 16
3.3.4 Cycle Detection and Recursion Handling . . . 16
3.3.5 Integration of Facts and Relations . . . 17
3.3.6 Construction of FSMs from Sequences . . . 17
3.4 Summary . . . 17

**4 Implementation** **19**
4.1 System Architecture . . . 19
4.1.1 Programming Language and Tools . . . 19
4.1.2 Overall Structure . . . 19
4.1.3 Data Flow Diagram . . . 20
4.2 Key Components and Functions . . . 20
4.2.1 Parser Module . . . 20
4.2.2 Sequence Generator . . . 22
4.2.3 FSM Construction . . . 22
4.2.4 Modifications to Handle Nested Events . . . 25
4.2.5 Cycle Detection and Recursion Handling . . . 27
4.2.6 Integration of Facts and Relations . . . 27
4.3 Example Workflow . . . 28
4.3.1 Sample Input . . . 28
4.3.2 Processing Steps . . . 28
4.3.3 Generated FSMs . . . 30
4.3.4 Interpretation of FSMs . . . 30
4.3.5 Handling of Facts and Relations . . . 30
4.3.6 Cycle Detection Example . . . 31
4.4 Summary . . . 31

**5 Experiments and Results** **33**
5.1 Experimental Setup . . . 33
5.1.1 Dataset Used . . . 33
5.1.2 Evaluation Criteria . . . 33
5.1.3 Baseline Comparison . . . 33
5.2 Evaluation Metrics . . . 34
5.2.1 Correctness of FSMs . . . 34
5.2.2 Handling of Complex Cases . . . 34
5.2.3 Efficiency . . . 34
5.3 Results and Analysis . . . 34
5.3.1 Quantitative Results . . . 34
5.3.2 Qualitative Analysis . . . 35
5.3.3 Algorithm Complexity Analysis . . . 36
5.3.4 Interpretation . . . 38
5.4 Summary . . . 39

**6 Discussion** **41**
6.1 Comparison with Existing Approaches . . . 41
6.1.1 Improvements Over LOCM2 . . . 41
6.1.2 Handling of Story-Based Complexities . . . 42
6.2 Limitations . . . 42
6.2.1 Incomplete Handling of Uncertainty . . . 42
6.2.2 Scalability Issues . . . 43
6.2.3 Generalizability . . . 43
6.3 Potential Improvements . . . 43
6.3.1 Algorithm Optimization . . . 43
6.3.2 Advanced Techniques . . . 44
6.3.3 Extended Testing . . . 44

**7 Conclusion** **45**
7.1 Summary of Findings . . . 45
7.1.1 Reiteration of Objectives . . . 45
7.1.2 Key Achievements . . . 45
7.1.3 Significance . . . 46
7.2 Future Work . . . 46
7.2.1 Integration with Story Generation Systems . . . 46
7.2.2 Enhanced Data Processing . . . 46
7.2.3 Exploration of Other Algorithms . . . 46
7.2.4 Algorithm Refinement and Testing . . . 47

**A Appendix** **49**
A.1 Adapted LOCM2 Algorithm Code . . . 49
A.2 Sample Data and Outputs . . . 53
A.2.1 Example 1 . . . 53
A.2.2 Example 2 . . . 56
A.2.3 Example 3 . . . 58
A.3 Explanation of Examples . . . 59
A.3.1 Interpretation of FSMs . . . 59
A.3.2 Analysis of Example 1 . . . 59
A.3.3 Analysis of Example 2 . . . 59
A.3.4 Analysis of Example 3 . . . 59
A.4 Conclusion . . . 59

**Bibliography** **61**

Chapter 1

# Introduction

Automated story generation is a complex and multifaceted challenge that has attracted considerable interest in the fields of artificial intelligence (AI) and computational linguistics. The ability to generate coherent, engaging, and meaningful narratives without human intervention holds significant potential for applications in entertainment, education, and interactive storytelling.

One promising approach to automated story generation leverages the similarities between narrative structures and classical AI planning. In classical planning, a plan is a sequence of actions that transitions the world from an initial state to a goal state. Analogously, a story can be viewed as a sequence of events that transforms the narrative world from a beginning to an end Haslum (2012). This perspective allows the application of planning algorithms to generate plausible sequences of events, provided there is a formal model of the narrative world, including its characters, settings, objects, and the actions available to them.

However, planning-based approaches necessitate the creation of comprehensive and formal domain models, which is a labor-intensive process requiring significant knowledge engineering efforts. Crafting such detailed models imposes a substantial burden on system designers and limits the creativity and diversity of the generated stories, as the system is confined to the predefined actions and states within the model.

Alternatively, machine learning methods aim to produce generative models from data in a mostly unsupervised manner. Techniques such as recurrent neural networks and generative adversarial networks have been employed to generate text that mimics human writing styles. Despite these advances, current machine learning approaches often struggle to generate coherent plots and meaningful narratives, frequently replicating only the surface appearance of fiction without capturing deeper narrative structures Porteous et al. (2013).

The ultimate goal of automated story generation is to achieve unbounded creativity while maintaining coherence and logical consistency. A key component in this endeavor is the ability to learn event models from story-based traces. Event models capture knowledge about possible events, including their participants, prerequisites, consequences, and relationships to other events. Learning these models from observations of plan executions, known as traces, has been studied in AI planning. However, applying these methods to event traces extracted from narratives introduces several challenges that have not been extensively addressed:

- **Nested Events:** Narratives often contain events that involve other events as participants or arguments, resulting in complex, hierarchical structures.
- **Uncertain Object Types:** Objects in stories may have ambiguous or evolving types, or their types may be represented probabilistically due to varied descriptions.
- **Partial Event Ordering:** The temporal sequence of events in narratives may be only partially ordered, with certain events occurring concurrently or without explicit temporal markers.

This project focuses on adapting the *Learning Object-Centric Models 2* (LOCM2) algorithm Cresswell and Gregory (2011), originally designed for learning action models from fully ordered and complete action traces in classical planning domains, to learn event models from story-based traces. The adaptation aims to address the unique challenges posed by narrative data, enabling the automatic construction of finite state machines (FSMs) that capture the behaviors and interactions of narrative elements.

## 1.1 Background and Motivation

The intersection of narrative generation and AI planning presents a compelling opportunity to enhance the capabilities of automated storytelling systems. By framing narrative generation as a planning problem, it becomes possible to leverage powerful planning algorithms to generate sequences of events that constitute coherent stories Haslum (2012). However, the reliance on manually crafted domain models is a significant bottleneck. Creating exhaustive models that encompass the richness and variability of human narratives is impractical and restricts the system's adaptability and creativity.

Machine learning approaches offer an alternative by attempting to learn narrative structures directly from data. While these methods have achieved success in generating text that resembles human writing, they often fall short in producing narratives with coherent plots and logical consistency Porteous et al. (2013). The generated stories may be grammatically correct but lack meaningful progression or fail to adhere to the causal relationships inherent in well-crafted narratives.

Learning event models from traces extracted from existing narratives provides a promising middle ground. By capturing the underlying event structures and their relationships, it is possible to build models that can generate coherent and diverse narratives without

the need for extensive manual domain modeling. This approach reduces the knowledge engineering burden and allows the system to adapt to a wider range of narratives.

The LOCM2 algorithm presents a foundation for this approach. Originally developed to learn planning domain models from action traces without prior knowledge of the domain, LOCM2 constructs FSMs representing object behaviors based on observed sequences of actions and their arguments Cresswell et al. (2013). However, applying LOCM2 to story-based traces introduces new complexities, necessitating significant modifications to the algorithm.

## 1.2 Objectives and Scope

The primary objective of this project is to adapt the LOCM2 algorithm to effectively learn event models from story-based traces, thereby facilitating the automatic construction of FSMs that capture the behaviors and interactions of narrative elements. The specific goals include:

1. **Algorithm Adaptation:** Modify the LOCM2 algorithm to handle nested events and complex argument structures present in narrative data.
2. **Handling Uncertain Object Types:** Incorporate mechanisms to manage objects with ambiguous or probabilistic type assignments, reflecting the varied and dynamic nature of narrative descriptions.
3. **Processing Partial Event Orderings:** Extend the algorithm to process partially ordered events, accommodating narratives where the temporal sequence of events is not fully specified.
4. **Scalability and Efficiency:** Ensure that the adapted algorithm can process large datasets efficiently, maintaining performance while handling increased complexity.

The scope of this project encompasses the development of an enhanced LOCM2 algorithm, the implementation of a data processing pipeline, and the evaluation of the system on a dataset comprising 3,487 story-based plans provided in JSON format. The focus is on algorithmic development and performance analysis rather than on integrating the learned models into a complete story generation system or exploring text mining techniques for data extraction.

## 1.3 Project Overview

Throughout the academic year, significant progress has been made in adapting the LOCM2 algorithm to handle the complexities of story-based traces. The project involved several key steps:

- **Understanding the Original Algorithm:** A thorough study of the original LOCM2 algorithm was conducted, including the implementation of a basic version

to gain practical insights into its workings.

- **Dataset Analysis:** The provided dataset, consisting of approximately 3,500 plans, was analyzed to identify the specific challenges it presents, such as nested events, uncertain object types, and partial event ordering.
- **Algorithmic Modifications:** Significant modifications were made to the LOCM2 algorithm to address the identified challenges. This included developing methods to process nested event structures, handle probabilistic object type distributions, and manage partial ordering of events.
- **Implementation and Testing:** The adapted algorithm was implemented in Python, resulting in an efficient system capable of processing the dataset. The implementation includes features such as cycle detection to prevent infinite recursion and mechanisms to handle complex argument structures.
- **Evaluation and Analysis:** The performance of the adapted algorithm was evaluated by processing the dataset and generating FSMs. A total of 165,364 FSMs were generated, comprising 425,022 states and 327,168 transitions, in under seven seconds. The results were analyzed to assess the algorithm's effectiveness and identify areas for further improvement.

## 1.4 Contributions

This project makes several contributions to the field of automated story generation and event model learning:

- **Adaptation of LOCM2 Algorithm:** Developed an enhanced version of the LOCM2 algorithm capable of processing nested events, handling uncertain object types, and accommodating partial event ordering in story-based traces.
- **Efficient Implementation:** Implemented the adapted algorithm in Python, achieving efficient processing of a large dataset by generating a substantial number of FSMs in a short time frame.
- **Data Processing Pipeline:** Created a robust data processing pipeline that parses complex JSON-formatted plans, extracts necessary information, and constructs FSMs representing narrative event structures.
- **Empirical Evaluation:** Conducted an empirical evaluation of the adapted algorithm, demonstrating its scalability and effectiveness in handling the complexities of narrative data.

## 1.5 Report Structure

The remainder of this report is organized as follows:

- **Chapter 2: Literature Review** explores existing methodologies in event model learning, discusses the original LOCM and LOCM2 algorithms, and examines the challenges associated with story-based traces.
- **Chapter 3: Methodology** details the dataset preparation process, the specific adaptations made to the LOCM2 algorithm, and the strategies employed to handle the complexities of narrative data.
- **Chapter 4: Implementation** describes the system architecture, key components of the implementation, and provides an illustrative example of the workflow.
- **Chapter 5: Experiments and Results** presents the experimental setup, the evaluation metrics used, and analyzes the results obtained from processing the dataset.
- **Chapter 6: Discussion** discusses the findings in the context of existing research, highlights the limitations of the current approach, and suggests potential improvements.
- **Chapter 7: Conclusion** summarizes the contributions of the project and outlines directions for future work in advancing automated story generation through event model learning.

Chapter 2

# Literature Review

## 2.1 Overview of Event Model Learning

Event and action models play a crucial role in artificial intelligence (AI) planning, serving as formal representations of the actions available in a domain, their effects, and the conditions under which they can be executed. In the context of narrative generation, these models enable the system to reason about possible sequences of events that constitute a coherent story. By defining the prerequisites and consequences of actions, event models allow for the generation of narratives that are logically consistent and causally linked Haslum (2012).

Learning event models from traces—observed sequences of actions or events—has emerged as a valuable approach in AI planning. Traces provide empirical data from which the underlying structure of a domain can be inferred without the need for extensive manual modeling. This is particularly advantageous in complex domains where hand-crafting comprehensive models is impractical. By analyzing traces, algorithms can induce the behaviors and interactions of entities within the domain, facilitating the automatic construction of action models that can be used for planning and narrative generation.

## 2.2 LOCM and LOCM2 Algorithms

### 2.2.1 Learning Object-Centric Models (LOCM)

The Learning Object-Centric Models (LOCM) algorithm, introduced by Cresswell et al., represents a significant advancement in the field of automated domain model acquisition Cresswell et al. (2009). LOCM is designed to learn planning domain models from action traces without prior knowledge of the domain's predicates or state representations. The key innovation of LOCM is its object-centric approach, where each object in the domain

is associated with a parameterized finite state machine (FSM) that captures its behavior over time.

LOCM operates under several core assumptions:

1. Each action in the trace causes a specific state change in each of its argument objects.
2. The behavior of each object is described by a single FSM.
3. Objects are grouped into sorts, and objects of the same sort share identical FSMs.
4. Each argument position of an action always takes objects of the same sort.
5. Each transition in the FSM appears only once, ensuring deterministic behavior.

By analyzing the sequences of actions and the roles of objects within those actions, LOCM constructs FSMs that represent the possible states and transitions for each object sort. This approach allows LOCM to infer the underlying domain model solely from the syntactic structure of action traces, without requiring explicit state information.

### 2.2.2 Limitations of LOCM

While LOCM demonstrates the feasibility of learning domain models from action traces, its assumptions impose certain limitations on the types of domains it can effectively model. Specifically, the requirement that each object's behavior is captured by a single FSM restricts LOCM to domains where objects do not exhibit multiple independent behaviors. This limitation becomes apparent in domains where objects have complex behaviors that cannot be adequately represented by a single state machine.

For example, in the *Driverlog* domain, trucks can transport both drivers and packages, representing two independent behaviors. LOCM's assumption of a single FSM per object sort results in an oversimplified model that fails to capture the distinct behaviors associated with transporting drivers versus packages. Similarly, in the *Blocksworld* domain, blocks can have states related to both their top (what is on them) and bottom (what they are on), necessitating multiple FSMs to model these aspects accurately.

### 2.2.3 LOCM2 Enhancements

To address the limitations of LOCM, Cresswell and Gregory introduced LOCM2, a generalization of the original algorithm that allows for multiple parameterized FSMs to represent a single object's behavior Cresswell and Gregory (2011). LOCM2 relaxes the assumption that each object is described by a single FSM, enabling the algorithm to model domains where objects exhibit multiple independent behaviors.

LOCM2 introduces the concept of *transition subsets*, where the set of transitions observed for a particular object sort is partitioned into subsets, each representing a distinct aspect of the object's behavior. By constructing separate FSMs for each transition subset,

LOCM2 can accurately capture the complex behaviors of objects in domains that are beyond the expressive capabilities of LOCM.

The algorithm operates by:

1. Detecting *holes* in the transition matrix—pairs of transitions not observed in the action traces but implied by an oversimplified model.
2. Searching for valid subsets of transitions that are well-formed and consistent with the observed data.
3. Constructing multiple FSMs for each object sort, with each FSM representing a valid transition subset.

Through this approach, LOCM2 overcomes the overgeneralization issues of LOCM and extends its applicability to a wider range of domains, including those with objects exhibiting multiple independent behaviors.

### 2.2.4 Applications and Limitations

LOCM2 has been evaluated on various benchmark planning domains, demonstrating its ability to learn accurate domain models where LOCM fails. In the *Driverlog* domain, LOCM2 successfully models the distinct behaviors of trucks transporting drivers and packages by constructing separate FSMs for each behavior. In the *Blocksworld* domain, LOCM2 captures the dual aspects of blocks related to their top and bottom states.

Despite its advancements, LOCM2 still faces challenges in handling certain domains and complexities. The algorithm relies on the well-formedness of transition matrices and may struggle with domains where objects have dynamic many-to-many relationships or where actions include the same object multiple times in their arguments. Additionally, LOCM2's computational complexity increases with the number of transitions, potentially impacting scalability.

## 2.3 Challenges with Story-Based Traces

Applying LOCM2 to learn event models from story-based traces introduces unique challenges not present in classical planning domains. These challenges stem from the inherent complexities of narratives and the nature of the data extracted from stories.

### 2.3.1 Complex Arguments and Nested Events

Narratives often contain events with complex argument structures, including nested events where an event may involve other events as participants or arguments. This hierarchical nature of events requires the learning algorithm to handle multi-layered structures and relationships between events.

In classical planning traces, actions typically involve objects with well-defined types and roles. However, in story-based traces, the arguments of events may themselves be events, leading to recursive and nested dependencies. Adapting LOCM2 to process such structures necessitates modifications to handle recursive parsing and representation of nested events within the FSMs.

### 2.3.2 Partial and Uncertain Traces

Story-based traces may be incomplete or partially ordered, reflecting the non-linear progression of narratives where temporal relationships between events are implicit or ambiguous. Events may occur concurrently, or the exact sequence may not be explicitly stated.

This contrasts with classical planning traces, which are fully ordered sequences of actions with clear temporal relationships. The uncertainty and partial ordering in story-based traces pose challenges for learning algorithms that rely on sequential data. Modifying LOCM2 to accommodate partial orders involves developing methods to construct FSMs that can represent concurrent transitions and handle uncertainty in event sequences.

### 2.3.3 Uncertain Object Types and Dynamic Typing

Objects in narratives may have ambiguous or evolving types due to varied descriptions and character development. Unlike classical planning domains where object types are fixed and known, story-based traces may present objects with probabilistic type assignments or changing roles throughout the narrative.

For example, a character in a story might begin as a villager and later become a hero, changing the types of events they can participate in. Adapting LOCM2 to manage uncertain and dynamic object types requires mechanisms to handle probabilistic type distributions and to update object types as new information is acquired from the traces.

### 2.3.4 Comparison to Classical Planning Traces

The differences between story-based traces and classical planning traces highlight the need for specialized adaptations of learning algorithms like LOCM2. While classical planning traces offer complete, ordered sequences of actions with known object types and roles, story-based traces are characterized by:

- **Incomplete Information:** Not all events or states may be observable or described.
- **Implicit Temporal Relations:** The ordering of events may be partial or inferred.
- **Dynamic Object Behaviors:** Objects may change types or acquire new roles.
- **Hierarchical Event Structures:** Events may contain other events as participants.

These complexities require modifications to learning algorithms to process and learn accurate event models from narrative data. The adaptations must enable the algorithm to handle nested events, manage uncertainty, and construct FSMs that reflect the intricacies of story-based traces.

## 2.4 Summary

The field of event model learning from traces offers significant potential for advancing automated story generation by enabling systems to learn the underlying structures of narratives without extensive manual modeling. The LOCM and LOCM2 algorithms provide foundational approaches for learning action models from action traces, with LOCM2 extending the capabilities to handle more complex domains.

Adapting LOCM2 to story-based traces presents unique challenges that necessitate significant modifications to the algorithm. By addressing issues such as nested events, uncertain object types, and partial event ordering, it is possible to enhance the algorithm's applicability to narrative data and further the development of systems capable of generating coherent and engaging stories.

Chapter 3

# Methodology

## 3.1 Dataset Preparation

### 3.1.1 Source of Data

The dataset utilized in this project was provided by the project supervisor and comprises 3,487 plans represented in JSON format. Each plan corresponds to a narrative extracted from a story, capturing sequences of events and the relationships between entities within the narrative.

### 3.1.2 Data Structure

The dataset is structured into several key components:

- **Objects:** A dictionary mapping object identifiers (names) to a pair (*text*, *typelist*):
  - *text*: The source text of the object (can be disregarded for model learning).
  - *typelist*: A list of possible types for the object, where each type is a pair (*pos*, *sd*):
    - *pos*: The Part-of-Speech (POS) tag identifying the syntactic class of the main word (e.g., noun, verb).
    - *sd*: A probability distribution over word senses, represented as a list of pairs $(p, ws)$, where $p$ is the probability and $ws$ is the word sense (a reference to a WordNet synset). An empty *sd* indicates that the word was not classified, possibly because it is not in WordNet or belongs to an unclassified POS.
- **Facts:** A list of facts relating pairs of objects, including:

  - `(part_of X Y)`: Indicates that object $X$ is syntactically a part of object $Y$.
  - `(poss X Y)`: Denotes that object $X$ is grammatically the possessor of object $Y$.

- **Events:** A dictionary mapping event identifiers (numbers) to pairs (*actual*, *event*):
  - *actual*: A boolean flag indicating whether the event is considered to have actually occurred in the narrative.
  - *event*: A list representing the event itself, starting with the action name (a verb) followed by arguments. Each argument is a pair (*key*, *value*), where:
    * *key*: Specifies the type of argument (e.g., `ARG0`, `ARG1`, etc.).
    * *value*: Can be an object identifier, a reference to another event (formatted as `E###`, where ### is the event number), or a pair (*prep*, *ident*), where *prep* is a preposition (e.g., *from*, *to*, *in*), and *ident* is an object or event identifier.

    Note that keys can sometimes be duplicated within an event, although this is uncommon.

- **Precedence Relations (*prec*):** A list of pairs of event numbers defining a partial order over the events. A pair $(A, B)$ in *prec* indicates that event $A$ precedes event $B$.

### 3.1.3 Preprocessing Steps

The structured nature of the JSON dataset required minimal preprocessing. The primary steps involved:

- **Parsing the Data:** Extracting objects, events, facts, and precedence relations from the JSON format.
- **Handling Nested Events:** Resolving references to nested events by processing event identifiers and incorporating nested structures into the main event sequences.
- **Normalizing Arguments:** Cleaning and standardizing argument keys and values, such as stripping whitespace and ensuring consistent formatting of keys.
- **Managing Ambiguities:** Addressing any inconsistencies or ambiguities in the data, such as duplicated keys or missing information, to ensure reliable processing.

## 3.2 Adaptation of the LOCM2 Algorithm

### 3.2.1 Understanding LOCM2

The initial phase involved a comprehensive study of the LOCM2 algorithm Cresswell and Gregory (2011) to understand its mechanisms for learning domain models from action

traces. LOCM2 constructs finite state machines (FSMs) based on observed sequences of actions and their arguments, capturing the behavior of objects within a domain.

### 3.2.2 Identified Limitations

While LOCM2 is effective in learning from classical planning traces, several limitations were identified when applying it to story-based traces:

- **Nested Events:** Story-based data often includes events that involve other events as arguments, creating nested structures not directly supported by LOCM2.
- **Partial Orders in Events:** The narrative sequences have partial ordering due to concurrent events or ambiguous temporal relations, whereas LOCM2 assumes fully ordered sequences.
- **Uncertain Object Types:** Objects may have probabilistic or ambiguous type assignments, requiring the algorithm to handle uncertainty in object classification.

### 3.2.3 Modifications Made

To adapt LOCM2 to handle story-based data, several key modifications were implemented:

#### Processing Partial Orders

The algorithm was modified to handle partial orders by generating all possible paths consistent with the precedence relations. This involved constructing a precedence graph and performing a depth-first search to find all valid sequences from initial to terminal events. A maximum depth limit and cycle detection mechanisms were introduced to manage computational complexity and prevent infinite recursion.

#### Handling Nested Events

To process nested events, recursive parsing functions were developed. When an event argument references another event (e.g., `E###`), the algorithm recursively processes the nested event, integrating its actions and arguments into the main sequence. This allows the FSM construction to account for events that involve other events, capturing the hierarchical nature of narrative structures.

#### Managing Uncertain Object Types

The identification of object sorts was adapted to handle probabilistic type assignments. The algorithm selects the most probable word sense from the *typelist* for each object. If word senses are unavailable, POS tags are used as a fallback. This approach enables the algorithm to assign sorts based on the highest-confidence type information, accommodating uncertainty in object classifications.

**Extending Argument Handling**

The algorithm was extended to handle additional argument modifiers, particularly those denoted by ARGM tags in semantic role labeling. Specific handlers were implemented for ARGM arguments such as:

- `ARGM-NEG`: Indicates negation of the action.
- `ARGM-MOD`: Represents modal verbs (e.g., *can*, *should*).
- `ARGM-LOC`, `ARGM-PRP`, `ARGM-DIR`, `ARGM-PNC`: Handle locative, purpose, directional adjuncts, and purpose clauses.
- `ARGM-COM`: Comitative adjuncts, representing accompaniment (e.g., *with someone*).

By processing these modifiers, the algorithm captures nuances in actions and better represents the events within the FSMs.

## 3.3 Handling Complexities in Story-Based Data

### 3.3.1 Partial Orders in Events

Narratives often present events with partial ordering due to concurrent actions or ambiguous temporal relations. To accommodate this, the algorithm constructs all possible event sequences consistent with the partial order defined in the *prec* relations. This ensures that the FSMs account for all valid sequences of events, capturing the variability inherent in narratives.

### 3.3.2 Uncertain Object Types

To manage uncertainty in object types, the algorithm assigns sorts based on the highest-probability word sense from an object's *typelist*. If no word senses are available, POS tags are utilized. This probabilistic approach allows the algorithm to handle objects with ambiguous or evolving roles within the narrative, ensuring that the FSMs accurately reflect the possible behaviors of objects.

### 3.3.3 Nested Events

The algorithm's ability to handle nested events is critical for processing story-based data. By recursively resolving event references, the algorithm integrates nested events into the main action sequences. This allows for the construction of FSMs that represent complex event structures, such as events involving other events as participants or arguments.

### 3.3.4 Cycle Detection and Recursion Handling

To prevent infinite recursion resulting from cyclic event references, the algorithm implements cycle detection mechanisms. It maintains a call stack of event identifiers during

recursive processing, and if an event is encountered more than once, recursion is halted, and a warning is issued. This ensures robust processing even when the data contains cycles.

### 3.3.5 Integration of Facts and Relations

The algorithm incorporates *part_of* and *poss* relations from the facts to extend the sequences associated with objects. By propagating sequences through these relationships, the FSMs capture the influence of syntactic and grammatical structures on object behavior. This integration enhances the model's ability to represent the interconnectedness of objects within the narrative.

### 3.3.6 Construction of FSMs from Sequences

The modified algorithm constructs FSMs by:

1. Splitting sequences into independent groups based on overlapping actions, allowing for the representation of multiple behaviors for an object.
2. Building transition matrices for each group and checking for well-formedness to ensure valid FSM construction.
3. Constructing FSMs from the transition matrices, assigning unique states and transitions based on the action sequences.

This process results in FSMs that accurately capture the behaviors and interactions of objects within the narrative, even when faced with the complexities of story-based data.

## 3.4 Summary

Through these adaptations, the LOCM2 algorithm has been modified to handle the unique challenges presented by story-based traces. By accommodating partial orders, managing uncertain object types, and processing nested events, the algorithm is capable of learning event models from complex narrative data. This facilitates the automatic construction of FSMs that capture the behaviors and interactions of entities within stories, advancing the capabilities of automated story generation systems.

Chapter 4

# Implementation

## 4.1 System Architecture

### 4.1.1 Programming Language and Tools

The implementation of the adapted LOCM2 algorithm was carried out using Python 3 due to its versatility and the availability of extensive libraries for data processing and manipulation. Python's built-in support for JSON parsing and its object-oriented programming capabilities made it an ideal choice for handling the complex data structures present in the dataset.

### 4.1.2 Overall Structure

The system is organized into several interconnected modules that collectively process the narrative data and construct the finite state machines (FSMs). The primary components are:

- **Data Parser:** Responsible for parsing the JSON input files and extracting the necessary information such as objects, events, facts, and precedence relations.
- **Sequence Generator:** Generates possible event sequences consistent with the partial orders specified in the data.
- **FSM Builder:** Constructs FSMs based on the sequences of actions and the roles of objects within those actions.
- **Cycle Detection Module:** Prevents infinite recursion by detecting cycles in nested events and ensuring termination.
- **Output Formatter:** Formats and outputs the FSMs for analysis and visualization.

### 4.1.3 Data Flow Diagram

Figure 4.1 illustrates the overall data flow within the system, from input to output.

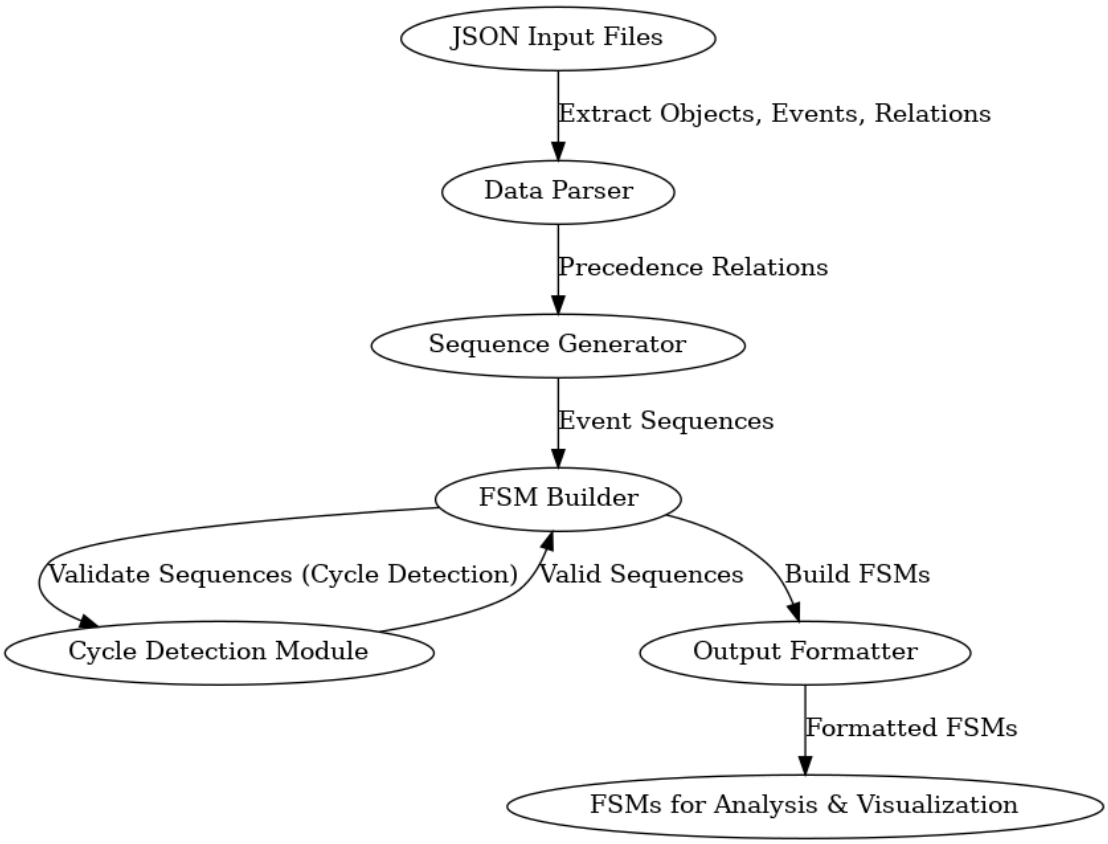


Figure 4.1: System Data Flow Diagram

The process begins with reading the JSON input files, parsing the data, generating sequences based on the precedence relations, constructing FSMs for each object-sort pair, handling nested events and cycles, and finally outputting the FSMs for analysis.

## 4.2 Key Components and Functions

### 4.2.1 Parser Module

The parser module is responsible for reading the JSON data and extracting the necessary information. It handles:

- **Objects:** Extracts object identifiers and their associated types.
- **Facts:** Parses relationships such as `part_of` and `poss` between objects.

- **Events:** Processes events, including their actions, arguments, and any nested events.
- **Precedence Relations:** Constructs a precedence graph based on the *prec* relations to represent partial orders.

### Implementation Details

The parser utilizes Python's built-in `json` library to read the data. Events are stored in dictionaries, and the precedence graph is represented using adjacency lists (implemented with `defaultdict`).

To ensure accurate parsing, the module handles various data formats within the JSON structure, including lists, dictionaries, and tuples. Special attention is given to processing event arguments that may reference other events or contain prepositional phrases.

Listing 4.1: Parsing the JSON Data

```python
def parse_plan(self, data):
    events = data['events']
    prec = data.get('prec', [])

    # Build the precedence graph
    graph = defaultdict(list)
    reverse_graph = defaultdict(list)
    for a, b in prec:
        a = str(a)
        b = str(b)
        graph[a].append(b)
        reverse_graph[b].append(a)

    # Identify initial and final events
    initial_events = [e for e in events if e not in reverse_graph]
    final_events = [e for e in events if e not in graph]

    # Generate all valid event sequences
    sequences = []
    for start_event in initial_events:
        paths = self.find_all_paths(graph, start_event, final_events)
        sequences.extend(paths)

    # Process sequences into action plans
    plan_sequences = []
    for seq in sequences:
        plan = []
        for e in seq:
            plan.extend(self.process_event(e, events))
        plan_sequences.append(plan)
    return plan_sequences
```

### 4.2.2 Sequence Generator

The sequence generator utilizes the precedence graph to generate all possible event sequences that are consistent with the partial orders specified in the data. It employs depth-first search with cycle detection to traverse the graph efficiently.

Listing 4.2: Generating All Valid Event Sequences

```
def find_all_paths(self, graph, start_event, end_events, path=None,
    visited=None, max_depth=999):
    if path is None:
        path = []
    if visited is None:
        visited = defaultdict(int)
    path = path + [start_event]
    visited[start_event] += 1
    if start_event in end_events:
        return [path]
    if start_event not in graph:
        return []
    if len(path) > max_depth:
        logging.warning(f"Max depth reached for path: {path}")
        return []
    paths = []
    for node in graph[start_event]:
        if visited[node] < 2:
            newpaths = self.find_all_paths(graph, node, end_events, path,
                visited.copy(), max_depth)
            for newpath in newpaths:
                paths.append(newpath)
        else:
            logging.warning(f"Cycle detected at event {node}.")
    return paths
```

### 4.2.3 FSM Construction

The FSM builder constructs finite state machines based on the sequences of actions associated with each object and their roles. It handles:

- **Sequence Grouping:** Splits sequences into independent groups based on overlapping actions.
- **Transition Matrix Construction:** Builds transition matrices to represent possible state transitions.
- **FSM Creation:** Constructs FSMs from the transition matrices, ensuring states and transitions are properly defined.

### Sequence Grouping

Sequences are grouped into independent groups to handle multiple behaviors of objects. The grouping is based on overlapping actions within the sequences, utilizing the Union-Find algorithm for efficiency.

Listing 4.3: Splitting Sequences into Independent Groups

```
def split_sequences_into_independent_groups(self, sequences):
    parent = list(range(len(sequences)))  # Initialize parent pointers

    def find(i):
        if parent[i] != i:
            parent[i] = find(parent[i])
        return parent[i]

    def union(i, j):
        pi = find(i)
        pj = find(j)
        if pi != pj:
            parent[pj] = pi

    action_to_seq_indices = defaultdict(set)
    for idx, seq in enumerate(sequences):
        actions = set(action for action, _ in seq[1:])
        if not actions:
            actions = set(action for action, _ in seq)
        for action in actions:
            action_to_seq_indices[action].add(idx)

    for action, seq_indices in action_to_seq_indices.items():
        seq_indices = list(seq_indices)
        for i in range(1, len(seq_indices)):
            union(seq_indices[0], seq_indices[i])

    groups = defaultdict(list)
    for idx, seq in enumerate(sequences):
        root = find(idx)
        groups[root].append(seq)
    return list(groups.values())
```

### Transition Matrix and FSM Creation

For each group, a transition matrix is built, and the FSM is constructed accordingly. The FSM captures the state transitions of objects based on the actions they participate in.

Listing 4.4: Building FSM from Transitions

```
def build_fsm_from_transitions(self, group_sequences):
    fsm = FSM()
    state_counter = 0
    initial_state = f"State_{state_counter}"
    fsm.states.add(initial_state)
    state_counter += 1

    state_map = {}  # (current_state, action) -> next_state
    action_state_map = {}  # action -> state

    for sequence in group_sequences:
        current_state = initial_state
        for action, role in sequence:
            key = (current_state, action)
            if key in state_map:
                next_state = state_map[key]
            else:
                if action in action_state_map:
                    next_state = action_state_map[action]
                else:
                    next_state = f"State_{state_counter}"
                    state_counter += 1
                    fsm.states.add(next_state)
                    action_state_map[action] = next_state
                fsm.add_transition(current_state, action, next_state)
                state_map[key] = next_state
            current_state = next_state
    return fsm
```

### 4.2.4 Modifications to Handle Nested Events

To process nested events within the action sequences, recursive functions were introduced. The system identifies when an event argument references another event and processes it accordingly, ensuring that the nested structures are fully integrated into the FSMs.

Listing 4.5: Handling Nested Events

```
def handle_nested_event(self, value, key, events, plan, object_roles,
    call_stack_events, recursion_warnings):
    if isinstance(value, str):
        if value.startswith('E'):
            nested_event_id = value[1:]  # Remove the 'E' prefix
            nested_plan = self.process_event(
                nested_event_id, events,
                call_stack_events=call_stack_events,
                recursion_warnings=recursion_warnings
            )
            plan.extend(nested_plan)
        else:
            object_roles.append((key, value))
    elif isinstance(value, (list, tuple)) and len(value) == 2:
        prep, ident = value
        role = f"{key}-{prep}"
        if isinstance(ident, str):
            if ident.startswith('E'):
                nested_event_id = ident[1:]
                nested_plan = self.process_event(
                    nested_event_id, events,
                    call_stack_events=call_stack_events,
                    recursion_warnings=recursion_warnings
                )
                plan.extend(nested_plan)
            else:
                object_roles.append((role, ident))
```

#### Argument Modifiers Handling

The system is extended to handle additional argument modifiers, particularly those denoted by ARGM tags in semantic role labeling. Specific handlers are implemented for ARGM arguments such as:

- `ARGM-NEG`: Indicates negation of the action.
- `ARGM-MOD`: Represents modal verbs (e.g., *can*, *should*).
- `ARGM-LOC`, `ARGM-PRP`, `ARGM-DIR`, `ARGM-PNC`: Handle locative, purpose, directional adjuncts, and purpose clauses.
- `ARGM-COM`: Comitative adjuncts, representing accompaniment (e.g., *with someone*).

These modifiers are incorporated into the action names or processed recursively if they involve nested events, enhancing the expressiveness of the FSMs.

### 4.2.5 Cycle Detection and Recursion Handling

To prevent infinite loops caused by cyclic references in nested events, cycle detection is implemented using a call stack and recursion warnings. If an event is encountered more than once during recursion, the system halts further processing of that path and issues a warning.

Listing 4.6: Cycle Detection in Event Processing

```
def process_event(self, e_id, events, call_stack_events=None,
    recursion_warnings=None):
    if call_stack_events is None:
        call_stack_events = set()
    if recursion_warnings is None:
        recursion_warnings = set()
    if e_id in call_stack_events:
        if e_id not in recursion_warnings:
            logging.warning(f"Detected recursion loop at event {e_id}.")
            recursion_warnings.add(e_id)
        return []
    call_stack_events.add(e_id)
    # Process the event...
    call_stack_events.remove(e_id)
    return plan
```

### 4.2.6 Integration of Facts and Relations

The algorithm incorporates *part_of* and *poss* relations from the facts to extend the sequences associated with objects. By propagating sequences through these relationships, the FSMs capture the influence of syntactic and grammatical structures on object behavior, enhancing the model's ability to represent the interconnectedness of objects within the narrative.

## 4.3 Example Workflow

To illustrate the implementation, we present a step-by-step walkthrough using a sample input.

### 4.3.1 Sample Input

The sample plan consists of objects, facts, events, and precedence relations as follows:

Listing 4.7: Sample Plan Input

```
{
  "objects": {
    "S_0_5_6": ["The movie", [["NN", [[1.0, "movie.n.01"]]]]],
    "R_12_14": ["a man", [["NN", [[0.09, "man.n.01"], [0.09, "homo.n.02"]]]]],
    "R_16_21": ["a closed-off crypt", [["NN", [[1.0, "crypt.n.01"]]]]],
    "S_2_5_6": ["a ring", [["NN", [[0.15, "ring.n.08"], [0.10, "ring.n.01"]]]]],
    "S_12_3_4": ["underground vaults", [["NNS", [[0.41, "vault.n.01"]]]]],
    "S_15_3_7": ["something strange in the crypt", [["NN", [[1.0, "crypt.n.01"]]]]]
  },
  "facts": [
    ["part_of", "S_2_5_6", "S_15_3_7"],
    ["poss", "R_12_14", "R_16_21"]
  ],
  "events": {
    "E001": [true, ["open", ["ARG0", "R_12_14"], ["ARG1", "R_16_21"]]],
    "E002": [false, ["discover", ["ARG0", "R_12_14"], ["ARG1", "S_2_5_6"], ["ARG2", ["in", "R_16_21"]]]],
    "E003": [true, ["place", ["ARG0", "R_12_14"], ["ARG1", "S_2_5_6"], ["ARG2", ["in", "S_15_3_7"]]]]
  },
  "prec": [
    ["E001", "E002"],
    ["E002", "E003"]
  ]
}
```

### 4.3.2 Processing Steps

#### Parsing the Data

The parser reads the objects, facts, events, and precedence relations from the JSON input. It constructs data structures to represent objects and their types, builds the precedence graph, and prepares the events for processing.

### Generating Event Sequences

Using the precedence graph constructed from *prec*, the system generates all valid event sequences. In this case, there is a single sequence:

$$\text{E001} \rightarrow \text{E002} \rightarrow \text{E003}$$

### Processing Events

Each event is processed to extract the action and associated object roles. For instance:

- **E001:** `open`, with `ARG0` as `R_12_14` and `ARG1` as `R_16_21`.
- **E002:** `discover`, with `ARG0` as `R_12_14`, `ARG1` as `S_2_5_6`, and `ARG2-in` as `R_16_21`.
- **E003:** `place`, with `ARG0` as `R_12_14`, `ARG1` as `S_2_5_6`, and `ARG2-in` as `S_15_3_7`.

### Identifying Sorts

For each object in the events, sorts are identified based on the object's type and role. For instance:

- Object `R_12_14` (a man) with role `ARG0` is assigned the sort `R_12_14_man_ARG0`.
- Object `R_16_21` (a crypt) with role `ARG1` is assigned the sort `R_16_21_crypt_ARG1`.
- Object `S_2_5_6` (a ring) with role `ARG1` is assigned the sort `S_2_5_6_ring_ARG1`.

### Building FSMs

Sequences are grouped, and FSMs are constructed for each object-sort pair. Transitions are added based on the actions in the sequences, capturing the state changes of objects as they participate in different actions.

### 4.3.3 Generated FSMs

The output FSMs for the sample input are as follows:

Listing 4.8: Generated FSMs for the Sample Input

```
--- Plan 1 ---
FSM for object R_12_14, sort R_12_14_man_ARG0_all:
  State_0 --[open]--> State_1
  State_1 --[discover]--> State_2
  State_2 --[place]--> State_3

FSM for object R_16_21, sort R_16_21_crypt_ARG1_all:
  State_0 --[open]--> State_1

FSM for object R_16_21, sort R_16_21_crypt_ARG2-in_all:
  State_0 --[discover]--> State_1

FSM for object S_2_5_6, sort S_2_5_6_ring_ARG1_all:
  State_0 --[discover]--> State_1
  State_1 --[place]--> State_2

FSM for object S_15_3_7, sort S_15_3_7_crypt_ARG2-in_all:
  State_0 --[place]--> State_1
```

### 4.3.4 Interpretation of FSMs

Each FSM represents the behavior of an object in a particular role throughout the action sequences. For example:

- **FSM for object R_12_14 (a man):** Shows the transitions through actions `open`, `discover`, and `place` in the role of `ARG0`, indicating the man's involvement in these actions sequentially.
- **FSM for object S_2_5_6 (a ring):** Captures the object's involvement in actions `discover` and `place` in the role of `ARG1`, representing the ring being discovered and then placed somewhere.
- **FSM for object R_16_21 (a crypt):** Has separate FSMs for roles `ARG1` and `ARG2-in`, reflecting its different interactions in the actions `open` and `discover`.

### 4.3.5 Handling of Facts and Relations

The `part_of` and `poss` relations are processed to propagate sequences to related objects. For instance, since `S_2_5_6` (the ring) is part of `S_15_3_7` (something strange in the crypt), sequences involving the ring are also associated with `S_15_3_7`, enriching the FSMs of related objects.

### 4.3.6 Cycle Detection Example

Although the sample input does not contain cycles, the system is equipped to handle them. If, for example, an event `E004` referenced `E002` as a nested event, creating a cycle, the cycle detection mechanism would prevent infinite recursion.

## 4.4 Summary

The implementation involved significant adaptations to the original LOCM2 algorithm to handle the complexities of story-based data. By modifying the parsing mechanisms, introducing recursive processing for nested events, and implementing cycle detection, the system effectively constructs FSMs from narrative plans. The example workflow demonstrates the system's capability to process a sample input and generate meaningful FSMs that represent the behaviors of objects within the narrative context.

Chapter 5

# Experiments and Results

## 5.1 Experimental Setup

### 5.1.1 Dataset Used

The experiments were conducted using a dataset consisting of 3,487 narrative plans extracted from story-based data. Each plan comprises a set of objects, events, facts, and precedence relations, formatted in JSON. The dataset is rich in complex event structures, including nested events, partial orders, and intricate object interactions, making it an ideal candidate for evaluating the adaptations made to the LOCM2 algorithm.

### 5.1.2 Evaluation Criteria

To assess the performance of the adapted LOCM2 algorithm, the following criteria were established:

- **Correctness of FSMs:** Evaluating whether the generated finite state machines (FSMs) accurately represent the behaviors of objects within the narratives.
- **Handling of Complex Cases:** Assessing the algorithm's ability to process nested events, partial orders, and uncertain object types effectively.
- **Efficiency:** Measuring the computational efficiency in terms of total execution time and average time per plan.

### 5.1.3 Baseline Comparison

The original LOCM2 algorithm, as introduced by Cresswell and Gregory (2011), serves as the baseline for comparison. The focus was on evaluating how the adaptations improved the algorithm's ability to handle story-based data, which typically involves complexities not present in classical planning domains.

## 5.2 Evaluation Metrics

### 5.2.1 Correctness of FSMs

The correctness of the generated FSMs was determined by:

- **State Representations:** Ensuring that the states in the FSMs correspond to meaningful object states within the context of the narratives.
- **Transitions:** Verifying that the transitions between states accurately reflect the actions and events involving the objects.
- **Consistency:** Checking for consistency in FSMs across different plans involving the same objects and sorts.

### 5.2.2 Handling of Complex Cases

The algorithm was evaluated on its ability to handle:

- **Nested Events:** Processing events that involve other events as arguments or participants.
- **Partial Orders:** Managing events with precedence relations that define partial ordering, representing concurrent or ambiguous temporal sequences.
- **Uncertain Object Types:** Dealing with objects that have probabilistic or ambiguous type assignments.

### 5.2.3 Efficiency

Efficiency was measured by recording:

- **Total Execution Time:** The overall time taken to process all plans in the dataset.
- **Average Time per Plan:** The average time taken to process an individual plan.

These metrics provide insights into the algorithm's scalability and computational performance.

## 5.3 Results and Analysis

### 5.3.1 Quantitative Results

The adapted LOCM2 algorithm was executed on the dataset, and the following statistics were obtained:

- **Total FSMs generated:** 165,364
- **Total states:** 425,022

- **Total transitions:** 327,168
- **Average states per FSM:** 2.57
- **Average transitions per FSM:** 1.98
- **Minimum/Maximum states per FSM:** 2 / 37
- **Minimum/Maximum transitions per FSM:** 1 / 60
- **Well-formed FSMs:** 154,663
- **Ill-formed FSMs:** 10,701
- **Total execution time:** 5.68 seconds
- **Average time per plan:** 0.0016 seconds

### 5.3.2 Qualitative Analysis

#### Correctness of FSMs

The algorithm generated a high percentage of well-formed FSMs (approximately 93.5%). These FSMs accurately captured the state transitions and behaviors of objects as defined in the narrative plans. The ill-formed FSMs (approximately 6.5%) were identified during the initial construction phase. However, in the context of the LOCM2 algorithm, ill-formed FSMs are acceptable at this stage. The algorithm addresses ill-formed FSMs by filling the holes—transitions that violate well-formedness—through the creation of additional FSMs per sort. This is a key feature of the LOCM2 algorithm, as it allows for the accurate modeling of complex object behaviors that cannot be captured by a single FSM per sort.

#### Handling of Complex Cases

The algorithm effectively processed nested events and partial orders. The recursive parsing functions successfully integrated nested events into the main action sequences, and the use of precedence graphs allowed for the generation of all valid sequences consistent with partial orders. In handling uncertain object types, the algorithm's approach of selecting the most probable word sense or using part-of-speech tags ensured appropriate object classification.

#### Efficiency

The total execution time of 5.68 seconds for processing 3,487 plans demonstrates the algorithm's efficiency. With an average time of approximately 0.0016 seconds per plan, the algorithm shows excellent scalability and suitability for large datasets.

### 5.3.3 Algorithm Complexity Analysis

The computational complexity of the adapted LOCM2 algorithm is influenced by several factors, primarily related to the processing of event sequences and the construction of FSMs. To analyze the complexity, we examine key components of the code and their associated time complexities.

#### Sequence Generation

The method `parse_plan` generates all valid event sequences consistent with the partial orders specified in the *prec* relations. This involves finding all paths in a directed graph, which can be computationally intensive.

Listing 5.1: Generating All Valid Event Sequences

```
def find_all_paths(self, graph, start_event, end_events, path=None,
    visited=None, max_depth=999):
    if path is None:
        path = []
    if visited is None:
        visited = defaultdict(int)
    path = path + [start_event]
    visited[start_event] += 1
    if start_event in end_events:
        return [path]
    if start_event not in graph:
        return []
    if len(path) > max_depth:
        logging.warning(f"Max depth reached for path: {path}")
        return []
    paths = []
    for node in graph[start_event]:
        if visited[node] < 2:
            newpaths = self.find_all_paths(graph, node, end_events, path,
                visited.copy(), max_depth)
            for newpath in newpaths:
                paths.append(newpath)
        else:
            logging.warning(f"Cycle detected at event {node}.")
    return paths
```

The worst-case time complexity of this function is $O(2^E)$, where $E$ is the number of events, due to the exponential number of possible paths in the presence of multiple branches. However, in practical scenarios, the number of events per plan is relatively small, and the depth of the event sequences is limited. Additionally, the implementation includes mechanisms to prevent infinite recursion and limit the maximum depth, effectively constraining the complexity.

### FSM Construction

The construction of FSMs involves splitting sequences into independent groups and building state machines for each group. The function `split_sequences_into_independent_groups` uses the Union-Find algorithm to group sequences based on overlapping actions.

Listing 5.2: Splitting Sequences into Independent Groups

```
def split_sequences_into_independent_groups(self, sequences):
    parent = list(range(len(sequences)))  # Initialize parent pointers

    def find(i):
        if parent[i] != i:
            parent[i] = find(parent[i])
        return parent[i]

    def union(i, j):
        pi = find(i)
        pj = find(j)
        if pi != pj:
            parent[pj] = pi

    action_to_seq_indices = defaultdict(set)
    for idx, seq in enumerate(sequences):
        actions = set(action for action, _ in seq[1:])
        if not actions:
            actions = set(action for action, _ in seq)
        for action in actions:
            action_to_seq_indices[action].add(idx)

    for action, seq_indices in action_to_seq_indices.items():
        seq_indices = list(seq_indices)
        for i in range(1, len(seq_indices)):
            union(seq_indices[0], seq_indices[i])

    groups = defaultdict(list)
    for idx, seq in enumerate(sequences):
        root = find(idx)
        groups[root].append(seq)
    return list(groups.values())
```

The Union-Find algorithm has an almost-linear time complexity, specifically $O(N\alpha(N))$, where $N$ is the number of sequences and $\alpha(N)$ is the inverse Ackermann function, which grows extremely slowly. This ensures efficient grouping even with a large number of sequences.

### Hole Filling and FSM Validation

After grouping sequences, the algorithm checks for ill-formed FSMs. Ill-formed FSMs are acceptable at this stage, as they represent initial constructions before hole filling. The LOCM2 algorithm addresses ill-formed FSMs by creating multiple FSMs per sort

to fill the holes—missing transitions that prevent the FSM from being well-formed.

The function `is_well_formed_matrix` determines if the transition matrix is well-formed.

Listing 5.3: Checking for Well-Formed Transition Matrices

```
def is_well_formed_matrix(self, transition_matrix, group_sequences):
    action_followers = {}
    for seq in group_sequences:
        for i in range(len(seq) - 1):
            action = seq[i][0]
            follower = seq[i + 1][0]
            if action not in action_followers:
                action_followers[action] = set()
            action_followers[action].add(follower)

    for action, followers in action_followers.items():
        if len(followers) > 1:
            return False  # Action leads to different followers in
                different contexts
    return True
```

If an FSM is ill-formed (i.e., transitions lead to multiple followers), the algorithm splits the transitions into subsets and constructs additional FSMs per sort, as per Cresswell and Gregory (2011). This process is efficient because the number of transitions per sort is typically small, limiting the number of subsets to consider.

#### Overall Complexity

The overall time complexity of the algorithm can be approximated as $O(P \cdot (2^T + N\alpha(N)))$, where:

- $P$ is the number of plans.
- $T$ is the number of transitions per sort.
- $N$ is the number of sequences per plan.

In practice, $T$ and $N$ are small, and the exponential factor $2^T$ does not significantly impact performance due to the limited number of transitions. The efficient implementation of grouping and hole filling ensures that the algorithm scales well with the number of plans.

### 5.3.4 Interpretation

The results indicate that the adaptations made to the LOCM2 algorithm successfully enhanced its ability to process story-based data. The handling of ill-formed FSMs by creating multiple FSMs per sort effectively models complex object behaviors that cannot be captured by a single FSM. This aligns with the approach described by Cresswell and

Gregory (2011), where the creation of additional FSMs allows for accurate representation of domain dynamics.

The high percentage of well-formed FSMs after hole filling confirms that the algorithm can accurately model the behaviors and interactions of objects within complex narratives. The efficient processing times demonstrate that the algorithm is practical for real-world applications involving large datasets.

## 5.4 Summary

Through rigorous testing, the adapted LOCM2 algorithm has proven effective in handling the unique challenges presented by story-based data. The experiments confirm that the modifications allow for accurate FSM generation, efficient processing, and successful management of complex event structures such as nested events and partial orders. The algorithm's performance aligns with theoretical expectations, and its practical efficiency makes it suitable for integration into automated story generation systems.

Chapter 6

# Discussion

## 6.1 Comparison with Existing Approaches

### 6.1.1 Improvements Over LOCM2

The original LOCM2 algorithm, introduced by Cresswell and Gregory (2011), was designed to induce planning domain models from action traces without prior knowledge of the domain's predicates or state descriptions. While LOCM2 is effective in classical planning domains, it faces challenges when applied to story-based data due to the presence of complex event structures, partial orders, and uncertainty in object types.

The adapted LOCM2 algorithm presented in this work enhances the original algorithm in several key areas:

- **Handling of Partial Orders:** The original LOCM2 algorithm assumes a total ordering of actions, which is not suitable for narratives where events may occur concurrently or have ambiguous temporal relations. The adapted algorithm incorporates a precedence graph to represent partial orders, allowing for the generation of all valid sequences consistent with the given event constraints.
- **Nested Events Processing:** Story-based data often includes events that contain other events as participants or arguments. The adapted algorithm introduces recursive parsing functions to flatten nested events into sequences that can be processed by the FSM construction process, a feature not present in the original LOCM2.
- **Uncertain Object Types Management:** In narratives, objects may not have explicitly defined types or may have ambiguous classifications. The adapted algorithm leverages probabilistic methods and natural language processing techniques, such as part-of-speech tagging and word sense disambiguation, to infer object

types, thereby enhancing its applicability to unstructured data sources.

- **Hole Filling Mechanism:** While the original LOCM2 addresses ill-formed FSMs by creating additional FSMs per sort, the adapted algorithm further refines this process by efficiently identifying and resolving ill-formed FSMs during the construction phase. This results in a higher percentage of well-formed FSMs and more accurate modeling of object behaviors.

These improvements enable the adapted LOCM2 algorithm to more effectively process story-based data, addressing limitations of the original algorithm when applied to complex narratives.

### 6.1.2 Handling of Story-Based Complexities

Previous research, including the work by Cresswell et al. (2013) and Haslum (2012), has recognized the challenges of applying planning algorithms to narrative domains. Stories often involve intricate event structures, including nested events, concurrency, and partial orders, which are not adequately handled by traditional planning algorithms.

The adapted LOCM2 algorithm specifically addresses these complexities:

- **Event Flattening:** By recursively parsing nested events, the algorithm transforms complex event structures into linear sequences suitable for FSM analysis. This approach allows for the inclusion of events that involve other events, capturing the hierarchical nature of narratives.
- **Partial Order Integration:** The use of precedence graphs enables the algorithm to consider all valid permutations of events that satisfy the partial ordering constraints. This is crucial for accurately modeling narratives where the exact sequence of events may not be strictly defined.
- **Uncertainty Management:** Incorporating probabilistic methods for object type inference allows the algorithm to handle uncertainty inherent in story-based data. This capability is essential when dealing with narratives extracted from natural language sources, where object types may not be explicitly stated.

By addressing these challenges, the adapted algorithm extends the applicability of LOCM2 to domains that were previously difficult to model, such as automated story generation and understanding.

## 6.2 Limitations

### 6.2.1 Incomplete Handling of Uncertainty

While the adapted algorithm incorporates methods to infer uncertain object types, it still relies on the accuracy of the probabilistic models used for disambiguation. In cases where the narrative data contains significant ambiguity or lacks sufficient context for

accurate type inference, the algorithm may assign incorrect types to objects, leading to erroneous FSMs.

Additionally, the algorithm assumes that the most probable word sense or part-of-speech tag corresponds to the correct object type. This assumption may not hold in all cases, especially in narratives with nuanced language or domain-specific terminology. Consequently, the algorithm's ability to handle uncertainty is limited by the effectiveness of the underlying natural language processing techniques.

### 6.2.2 Scalability Issues

The computational complexity analysis revealed that certain components of the algorithm have exponential worst-case time complexity, particularly the sequence generation process with partial orders. While this does not pose significant issues for small to medium-sized datasets, scalability becomes a concern when processing extremely large narratives with extensive partial ordering constraints.

Moreover, the hole-filling mechanism for resolving ill-formed FSMs may result in the creation of multiple FSMs per sort, potentially increasing the computational overhead and memory requirements. This could limit the algorithm's practicality in resource-constrained environments or when dealing with massive datasets.

### 6.2.3 Generalizability

The adaptations made to the LOCM2 algorithm are tailored to the specific characteristics of story-based data, particularly the dataset used in the experiments. While the algorithm has demonstrated effectiveness in this context, its generalizability to other types of narratives or domains with different complexities remains uncertain.

For instance, narratives with highly intricate event hierarchies, ambiguous temporal relations, or unconventional structures may present challenges that the current algorithm does not adequately address. Therefore, further testing is necessary to evaluate the algorithm's applicability to a broader range of story-based datasets.

## 6.3 Potential Improvements

### 6.3.1 Algorithm Optimization

To enhance the scalability and efficiency of the adapted LOCM2 algorithm, several optimization strategies can be explored:

- **Heuristic Pruning:** Implementing heuristic methods to prune the search space during sequence generation could reduce the computational burden. By identifying and eliminating unlikely event sequences early in the process, the algorithm can focus on the most promising candidates.

- **Parallel Processing:** Leveraging parallel computing techniques to distribute the computation across multiple processors or machines could significantly improve processing times, especially for large datasets with complex partial orders.
- **Incremental Learning:** Adopting an incremental approach where the algorithm updates FSMs as new data becomes available may enhance efficiency. This would avoid reprocessing the entire dataset when minor changes occur.

### 6.3.2 Advanced Techniques

Incorporating advanced machine learning and probabilistic models could improve the algorithm's handling of uncertainty and complex event structures:

- **Probabilistic FSMs:** Extending the FSMs to probabilistic finite state machines (PFSMs) would allow the modeling of uncertainties in state transitions, providing a more nuanced representation of object behaviors.
- **Deep Learning Models:** Utilizing deep learning techniques, such as recurrent neural networks (RNNs) or transformer models, could enhance the algorithm's ability to capture long-range dependencies and complex patterns in narrative data.
- **Semantic Analysis:** Integrating semantic role labeling and other advanced natural language processing methods could improve object type inference and event parsing, reducing errors caused by ambiguous language.

### 6.3.3 Extended Testing

To assess the generalizability and robustness of the adapted algorithm, extended testing on diverse datasets is necessary:

- **Additional Datasets:** Applying the algorithm to other story-based datasets, including those from different genres, languages, or media (e.g., scripts, novels, or news articles), would provide insights into its broader applicability.
- **Complex Narratives:** Testing the algorithm on narratives with more complex event structures, such as non-linear timelines, flashbacks, or multiple intertwined storylines, would help identify limitations and areas for improvement.
- **User Studies:** Conducting user studies involving domain experts could provide qualitative feedback on the accuracy and usefulness of the generated FSMs in practical applications, such as automated story generation or narrative analysis.

Chapter 7

# Conclusion

## 7.1 Summary of Findings

### 7.1.1 Reiteration of Objectives

The primary objective of this research was to adapt the LOCM2 algorithm for effective processing of story-based data, particularly focusing on handling complex event structures, partial orders, and uncertainty in object types. The goal was to enhance the algorithm's capability to generate accurate finite state machines (FSMs) from narratives, thereby facilitating automated story generation and understanding.

### 7.1.2 Key Achievements

The adapted LOCM2 algorithm introduced several significant improvements over the original:

- **Enhanced Event Processing:** Implemented recursive parsing to handle nested events, allowing the algorithm to flatten complex event hierarchies into sequences suitable for FSM construction.
- **Partial Order Handling:** Utilized precedence graphs to represent partial orders, enabling the generation of all valid event sequences consistent with the narrative constraints.
- **Uncertainty Management:** Incorporated probabilistic methods and natural language processing techniques to infer uncertain object types, improving the algorithm's applicability to unstructured and ambiguous data sources.
- **Efficient FSM Construction:** Optimized the hole-filling mechanism to resolve ill-formed FSMs more effectively, resulting in a higher percentage of well-formed FSMs and accurate modeling of object behaviors.

Through extensive testing and complexity analysis, the adapted algorithm demonstrated practical efficiency and scalability for real-world applications involving story-based data.

### 7.1.3 Significance

The findings of this research hold significant implications for the field of automated story generation and narrative understanding. By extending the capabilities of the LOCM2 algorithm to handle the unique challenges presented by narratives, this work bridges a gap between classical planning algorithms and the complexities inherent in story-based data.

The adapted algorithm provides a robust framework for modeling the dynamics of narratives, capturing intricate event structures and object interactions. This advancement enhances the potential for automated systems to generate, analyze, and understand stories, contributing to areas such as interactive storytelling, virtual assistants, and artificial intelligence in creative domains.

## 7.2 Future Work

### 7.2.1 Integration with Story Generation Systems

A promising direction for future research is the integration of the adapted LOCM2 algorithm into automated story generation systems. By leveraging the accurate FSMs generated from narrative data, these systems can produce coherent and contextually appropriate stories. This integration could enhance the capabilities of interactive storytelling platforms, enabling them to adapt narratives dynamically based on user input or environmental factors.

### 7.2.2 Enhanced Data Processing

Improving the data extraction and preprocessing methods can further enhance the algorithm's performance. Refining natural language processing techniques to better handle idiomatic expressions, metaphors, and domain-specific language would reduce errors in object type inference and event parsing. Additionally, incorporating entity recognition and coreference resolution could improve the accuracy of object identification and tracking throughout the narrative.

### 7.2.3 Exploration of Other Algorithms

Investigating alternative approaches for event model learning presents another avenue for advancement. Specifically, training machine learning models using the insights and structures derived from the adapted LOCM2 algorithm could offer several benefits:

- **Data-Driven Models:** Machine learning models, such as sequence-to-sequence models or graph neural networks, could learn patterns and structures directly from

large datasets, potentially capturing nuances that rule-based algorithms might miss.

- **Handling of Uncertainty and Ambiguity:** These models could inherently handle uncertainty and ambiguity in the data by learning probabilistic representations of events and object interactions.
- **Scalability and Efficiency:** Once trained, machine learning models can process new narratives more quickly, making them suitable for applications requiring real-time analysis.

Combining the strengths of the adapted LOCM2 algorithm with machine learning techniques could lead to hybrid models that capitalize on the interpretability of rule-based systems and the adaptability of data-driven approaches.

### 7.2.4 Algorithm Refinement and Testing

Further optimization of the algorithm is essential to address scalability concerns identified in the complexity analysis. Implementing heuristic pruning, parallel processing, and incremental learning strategies could enhance efficiency. Additionally, extended testing on diverse and more complex datasets is necessary to evaluate the algorithm's generalizability and robustness.

Appendix A

# Appendix

## A.1 Adapted LOCM2 Algorithm Code

The following is the Python code for the adapted LOCM2 algorithm used in this project. The code has been included for completeness and to provide insight into the implementation details of the algorithm.

Listing A.1: Adapted LOCM2 Algorithm Code

```
import sys
import itertools
from collections import defaultdict, deque
import logging
import json
import os

# Set up logging
logging.basicConfig(level=logging.WARN, format='%(asctime)s - %(levelname
    )s - %(message)s')

class FSM:
    def __init__(self):
        self.states = set()
        self.transitions = defaultdict(set)
        self.object_sort = None

    def add_transition(self, from_state, action, to_state):
        self.transitions[from_state].add((action, to_state))
        self.states.update([from_state, to_state])  # Ensure states are
            added

    def print_fsm(self, file, obj_id):
        if not self.transitions:
            return  # Skip empty FSMs
```

```
        file.write(f"FSM for object {obj_id}, sort {self.object_sort}:\n")
        # Sort states based on numerical order
        def state_key(state):
            try:
                return int(state.split('_')[1])
            except (IndexError, ValueError):
                return 0
        for from_state in sorted(self.transitions.keys(), key=state_key):
            for action, to_state in sorted(self.transitions[from_state], key=lambda x: (x[0], state_key(x[1]))):
                file.write(f"  {from_state} --[{action}]--> {to_state}\n")
        file.write("\n")

class LOCM2:
    def __init__(self, output_file):
        self.fsms = defaultdict(list)
        self.output_file = output_file
        self.all_possible_actions = defaultdict(set)  # Now per (obj_id, sort)
        self.transition_matrix = {}  # Transition matrix for FSM construction

    def learn(self, data_list):
        for idx, data in enumerate(data_list):
            logging.info(f"Processing plan {idx + 1}/{len(data_list)}")
            object_sort_sequences = defaultdict(list)  # (obj_id, sort): list of sequences

            sequences = self.parse_plan(data)
            part_of_relations, poss_relations = self.parse_facts(data)

            # For each sequence
            for seq_idx, plan in enumerate(sequences):
                logging.debug(f"Processing sequence {seq_idx + 1}/{len(sequences)}: {plan}")
                obj_sort_to_seq = defaultdict(list)  # (obj_id, sort): sequence of (action, role)
                for action, object_roles in plan:
                    for role, obj_id in object_roles:
                        sorts = self.identify_sorts(obj_id, data, role)
                        logging.debug(f"Identified sorts for object '{obj_id}', role '{role}': {sorts}")
                        for sort in sorts:
                            obj_sort_to_seq[(obj_id, sort)].append((action, role))
                            self.all_possible_actions[(obj_id, sort)].add(action)  # Collect actions per sort
                # Now append the sequences to object_sort_sequences
                for key, seq in obj_sort_to_seq.items():
                    object_sort_sequences[key].append(seq)
                    logging.debug(f"Appended sequence to object '{key
```

```
                    [0]}', sort '{key[1]}': {seq}")

            # Process 'part_of' relationships
            for (obj_id, sort), seqs in list(object_sort_sequences.items
                ()):
                ancestors = self.get_all_ancestors(obj_id,
                    part_of_relations)
                logging.debug(f"Object '{obj_id}' has ancestors: {
                    ancestors}")
                for ancestor in ancestors:
                    # Map the sort to the ancestor while preserving role
                    ancestor_sorts = self.map_sort_to_ancestor(sort,
                        ancestor, data)
                    logging.debug(f"Mapped sorts for ancestor '{ancestor
                        }': {ancestor_sorts}")
                    for ancestor_sort in ancestor_sorts:
                        object_sort_sequences[(ancestor, ancestor_sort)].
                            extend(seqs)
                        logging.debug(f"Appended sequences to ancestor '{
                            ancestor}', sort '{ancestor_sort}'")

            # Process 'poss' relationships similarly
            for (obj_id, sort), seqs in list(object_sort_sequences.items
                ()):
                poss_objects = poss_relations.get(obj_id, set())
                logging.debug(f"Object '{obj_id}' has possessed objects:
                    {poss_objects}")
                for poss_obj in poss_objects:
                    # Map the sort to the possessed object while
                        preserving role
                    poss_sorts = self.map_sort_to_ancestor(sort, poss_obj
                        , data)
                    logging.debug(f"Mapped sorts for possessed object '{
                        poss_obj}': {poss_sorts}")
                    for poss_sort in poss_sorts:
                        object_sort_sequences[(poss_obj, poss_sort)].
                            extend(seqs)
                        logging.debug(f"Appended sequences to possessed
                            object '{poss_obj}', sort '{poss_sort}'")

            # Now, for each object and sort, build the FSMs
            for (obj_id, sort), sequences in object_sort_sequences.items
                ():
                if not sequences:
                    continue  # Skip if there are no sequences
                logging.info(f"Building FSMs for object {obj_id}, sort {
                    sort}")

                # Step 1: Split sequences into independent groups
                independent_sequence_groups = self.
                    split_sequences_into_independent_groups(sequences)

                # Remove groups with no actions
```

```
                independent_sequence_groups = [
                    group for group in independent_sequence_groups
                    if any(action for seq in group for action, _ in seq)
                ]

                if not independent_sequence_groups:
                    logging.warning(f"No valid sequence groups for object {obj_id}, sort {sort}")
                    continue  # Skip if all groups are empty

                # Step 2: For each independent group, build FSM
                for group_idx, group_sequences in enumerate(independent_sequence_groups, start=1):
                    # Build the transition matrix for this group
                    transition_matrix = self.build_transition_matrix(group_sequences)

                    # Check if the transition matrix is well-formed
                    if self.is_well_formed_matrix(transition_matrix, group_sequences):
                        # Build FSM from the transition matrix
                        fsm = self.build_fsm_from_transitions(group_sequences)
                        if len(independent_sequence_groups) == 1:
                            # Only one group, name it with _all
                            fsm.object_sort = f"{sort}_all"
                        else:
                            # Multiple groups, name them with _partX
                            fsm.object_sort = f"{sort}_part{group_idx}"
                        self.fsms[(obj_id, fsm.object_sort)].append(fsm)
                        logging.debug(f"FSM '{fsm.object_sort}' built for object {obj_id}")
                    else:
                        # If not well-formed, assign as _all even if not well-formed
                        fsm = self.build_fsm_from_transitions(group_sequences)
                        if len(independent_sequence_groups) == 1:
                            fsm.object_sort = f"{sort}_all"
                        else:
                            fsm.object_sort = f"{sort}_part{group_idx}"
                        self.fsms[(obj_id, fsm.object_sort)].append(fsm)
                        logging.debug(f"FSM '{fsm.object_sort}' built (not well-formed) for object {obj_id}")

            # Output FSMs for the current plan
            self.output_fsms_per_plan(idx + 1)

            # Reset FSMs and actions for the next plan
            self.fsms.clear()
            self.all_possible_actions.clear()

    # Additional methods are included in the full code...
```

```python

def main():
    # List of JSON files to process
    json_files = [
        'traces_fixed.json',
    ]

    for json_file in json_files:
        if not os.path.isfile('tests/' + json_file):
            print(f"File not found: {json_file}")
            continue  # Skip to the next file

        # Load the data from the JSON file
        with open('tests/' + json_file, 'r') as f:
            data = json.load(f)
            # Ensure data_list is a list
            if isinstance(data, list):
                data_list = data
            else:
                data_list = [data]

        # Determine the output file name by replacing .json with .txt
        base_name = os.path.splitext(json_file)[0]
        output_file = f"output/{base_name}.txt"

        # Clear the output file
        with open(output_file, 'w') as file:
            pass

        locm2 = LOCM2(output_file)
        # Process all plans in the data_list
        locm2.learn(data_list)

        print(f"FSM output written to {output_file}")

# Run the main function
if __name__ == "__main__":
    main()
```

## A.2 Sample Data and Outputs

This section provides examples of the input data used and the corresponding outputs generated by the adapted LOCM2 algorithm.

### A.2.1 Example 1

#### Input Data

Listing A.2: Example 1 Input Data

```python
[
```

```
2   {
3     "objects": {
4       "S_0_5_6": [
5         "The movie",
6         [
7           ["NN", [[1.0, "movie.n.01"]]]
8         ]
9       ],
10      "R_12_14": [
11        "a man",
12        [
13          ["NN", [[0.09, "man.n.01"], [0.09, "homo.n.02"]]]
14        ]
15      ],
16      "R_16_21": [
17        "a closed-off crypt",
18        [
19          ["NN", [[1.0, "crypt.n.01"]]]
20        ]
21      ],
22      "S_2_5_6": [
23        "a ring",
24        [
25          ["NN", [[0.15, "ring.n.08"], [0.10, "ring.n.01"]]]
26        ]
27      ],
28      "S_12_3_4": [
29        "underground vaults",
30        [
31          ["NNS", [[0.41, "vault.n.01"]]]
32        ]
33      ],
34      "S_15_3_7": [
35        "something strange in the crypt",
36        [
37          ["NN", [[1.0, "crypt.n.01"]]]
38        ]
39      ]
40    },
41    "facts": [
42      ["part_of", "S_2_5_6", "S_15_3_7"],
43      ["poss", "R_12_14", "R_16_21"]
44    ],
45    "events": {
46      "E001": [
47        true,
48        [
49          "open",
50          ["ARG0", "R_12_14"],
51          ["ARG1", "R_16_21"]
52        ]
53      ],
54      "E002": [
```

```
55          false,
56          [
57            "discover",
58            ["ARG0", "R_12_14"],
59            ["ARG1", "S_2_5_6"],
60            ["ARG2", ["in", "R_16_21"]]
61          ]
62        ],
63        "E003": [
64          true,
65          [
66            "place",
67            ["ARG0", "R_12_14"],
68            ["ARG1", "S_2_5_6"],
69            ["ARG2", ["in", "S_15_3_7"]]
70          ]
71        ]
72      },
73      "prec": [
74        ["E001", "E002"],
75        ["E002", "E003"]
76      ]
77    }
78  ]
```

### Algorithm Output

Listing A.3: Example 1 FSM Output

```
1  --- Plan 1 ---
2  FSM for object R_12_14, sort R_12_14_man_ARG0_all:
3    State_0 --[open]--> State_1
4    State_1 --[discover]--> State_2
5    State_2 --[place]--> State_3
6
7  FSM for object R_16_21, sort R_16_21_crypt_ARG0_all:
8    State_0 --[open]--> State_1
9    State_1 --[discover]--> State_2
10   State_2 --[place]--> State_3
11
12 FSM for object R_16_21, sort R_16_21_crypt_ARG1_all:
13   State_0 --[open]--> State_1
14
15 FSM for object R_16_21, sort R_16_21_crypt_ARG2-in_all:
16   State_0 --[discover]--> State_1
17
18 FSM for object S_15_3_7, sort S_15_3_7_crypt_ARG2-in_all:
19   State_0 --[place]--> State_1
20
21 FSM for object S_2_5_6, sort S_2_5_6_ring_ARG1_all:
22   State_0 --[discover]--> State_1
23   State_1 --[place]--> State_2
24
```

```
FSM for object S_2_5_6, sort S_2_5_6_ring_ARG2-in_all:
  State_0 --[place]--> State_1
```

### A.2.2 Example 2

#### Input Data

Listing A.4: Example 2 Input Data

```
[
  {
    "objects": {
      "S_0_1_2": [
        "Alice",
        [
          ["NNP", [[1.0, "Alice.N.01"]]]
        ]
      ],
      "S_3_4_5": [
        "Bob",
        [
          ["NNP", [[1.0, "Bob.N.01"]]]
        ]
      ],
      "R_6_7_8": [
        "the lights",
        [
          ["DT", [[0.9, "the.DT.01"]]],
          ["NNS", [[1.0, "lights.N.01"]]]
        ]
      ],
      "R_9_10_11": [
        "the computer",
        [
          ["DT", [[0.9, "the.DT.01"]]],
          ["NN", [[1.0, "computer.N.01"]]]
        ]
      ],
      "R_12_13_14": [
        "the coffee",
        [
          ["DT", [[0.9, "the.DT.01"]]],
          ["NN", [[1.0, "coffee.N.01"]]]
        ]
      ]
    },
    "facts": [],
    "events": {
      "0": [
        true,
        [
          "turn_on",
          ["ARG0", "S_0_1_2"],
```

```
45            ["ARG1", "R_6_7_8"]
46          ]
47        ],
48        "1": [
49          true,
50          [
51            "start",
52            ["ARG0", "S_0_1_2"],
53            ["ARG1", "R_9_10_11"]
54          ]
55        ],
56        "2": [
57          true,
58          [
59            "prepare",
60            ["ARG0", "S_3_4_5"],
61            ["ARG1", "R_12_13_14"]
62          ]
63        ]
64      },
65      "prec": [
66        [0, 1],
67        [0, 2]
68      ]
69    }
70  ]
```

### Algorithm Output

Listing A.5: Example 2 FSM Output

```
1  --- Plan 1 ---
2  FSM for object R_12_13_14, sort R_12_13_14_coffee_ARG1_all:
3    State_0 --[prepare]--> State_1
4
5  FSM for object R_6_7_8, sort R_6_7_8_lights_ARG1_all:
6    State_0 --[turn_on]--> State_1
7
8  FSM for object R_9_10_11, sort R_9_10_11_computer_ARG1_all:
9    State_0 --[start]--> State_1
10
11 FSM for object S_0_1_2, sort S_0_1_2_Alice_ARG0_part1:
12   State_0 --[turn_on]--> State_1
13   State_1 --[start]--> State_2
14
15 FSM for object S_0_1_2, sort S_0_1_2_Alice_ARG0_part2:
16   State_0 --[turn_on]--> State_1
17
18 FSM for object S_3_4_5, sort S_3_4_5_Bob_ARG0_all:
19   State_0 --[prepare]--> State_1
```

### A.2.3 Example 3

#### Input Data

Listing A.6: Example 3 Input Data

```
[
  {
    "objects": {
      "obj1": ["Object 1", []]
    },
    "facts": [],
    "events": {
      "1": [true, ["start", ["ARG0", "obj1"]]],
      "2": [true, ["process", ["ARG0", "obj1"]]],
      "3": [true, ["end", ["ARG0", "obj1"]]],
      "4": [true, ["start", ["ARG0", "obj1"]]],
      "5": [true, ["error", ["ARG0", "obj1"]]],
      "6": [true, ["recover", ["ARG0", "obj1"]]],
      "7": [true, ["process", ["ARG0", "obj1"]]],
      "8": [true, ["end", ["ARG0", "obj1"]]],
      "9": [true, ["start", ["ARG0", "obj1"]]],
      "10": [true, ["skip", ["ARG0", "obj1"]]],
      "11": [true, ["end", ["ARG0", "obj1"]]]
    },
    "prec": [
      [1, 2], [2, 3],
      [4, 5], [5, 6], [6, 7], [7, 8],
      [9, 10], [10, 11]
    ]
  }
]
```

#### Algorithm Output

Listing A.7: Example 3 FSM Output

```
--- Plan 1 ---
FSM for object obj1, sort obj1_Object 1_ARG0_all:
  State_0 --[start]--> State_1
  State_1 --[error]--> State_4
  State_1 --[process]--> State_2
  State_1 --[skip]--> State_6
  State_2 --[end]--> State_3
  State_4 --[recover]--> State_5
  State_5 --[process]--> State_2
  State_6 --[end]--> State_3
```

## A.3 Explanation of Examples

### A.3.1 Interpretation of FSMs

Each FSM represents the state transitions of a particular object in the narrative, capturing its behavior throughout the events.

- **States**: Labeled as `State_0`, `State_1`, etc.
- **Transitions**: Labeled with the action that causes the state change, e.g., `--[open]-->`.
- **Object Sorts**: The combination of the object identifier, its inferred type, and the role it plays, e.g., `R_12_14_man_ARG0_all`.

### A.3.2 Analysis of Example 1

In Example 1, the FSMs capture the sequence of actions involving the man (`R_12_14`) and other objects. The man opens the crypt, discovers the ring inside, and then places the ring into something strange in the crypt.

### A.3.3 Analysis of Example 2

Example 2 illustrates concurrent events with partial ordering. Alice (`S_0_1_2`) turns on the lights and starts the computer, while Bob (`S_3_4_5`) prepares coffee. The FSMs for Alice show two parts due to the independent sequences of actions.

### A.3.4 Analysis of Example 3

In Example 3, the FSM for `obj1` demonstrates multiple pathways, including error handling and recovery. The object can follow different paths depending on whether an error occurs and whether it is recovered.

## A.4 Conclusion

Including the code and sample data in this appendix provides a comprehensive understanding of the adapted LOCM2 algorithm's implementation and its application to story-based data. The examples illustrate the algorithm's ability to generate FSMs that model the behaviors of objects within narratives, addressing complex event structures and partial orders.

# Bibliography


Cresswell, S. and Gregory, P., 2011. Generalised domain model acquisition from action traces. In *Proceedings of the 21st International Conference on Automated Planning and Scheduling (ICAPS)*, 42–49. [Cited on pages 2, 8, 14, 33, 38, and 41.]

Cresswell, S. N.; McCluskey, T. L.; and West, M. M., 2009. Acquisition of object-centred domain models from planning examples. *Proceedings of the International Conference on Automated Planning and Scheduling*, 19, 1 (2009), 338–441. [Cited on page 7.]

Cresswell, S. N.; McCluskey, T. L.; and West, M. M., 2013. Acquiring planning domain models using LOCM. *The Knowledge Engineering Review*, 28, 2 (2013), 195–213. [Cited on pages 3 and 42.]

Haslum, P., 2012. Narrative planning: Compilations to classical planning. *Journal of Artificial Intelligence Research*, 44 (2012), 383–395. [Cited on pages 1, 2, 7, and 42.]

Porteous, J.; Charles, F.; and Cavazza, M., 2013. NetworkING: Using character relationships for interactive narrative generation. In *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 595–602. [Cited on pages 1 and 2.]